

OCR - PROJET 8

Construisez et testez une infrastructure de données

Mathilde LE SOLLIEC
02-2026

SOMMAIRE

- 1 Contexte
- 2 Architecture de base de données
- 3 Logigramme
- 4 Résultat
- 5 Qualité de données obtenues

1. Contexte

CONTEXTE

Entreprise

GreenAndCoop, **fournisseur coopératif d'électricité renouvelable** dans les Hauts-de-France.

Projet

Initiative Data Science visant à améliorer la prévision de la demande d'électricité.

Problématique

Manque de relevés de stations météorologiques officielles dans certains zones

Sources de données nouvelles

Stations météorologiques semi-professionnelles et amateurs (InfoClimat, Weather Underground).

OBJECTIF DE LA MISSION



Centraliser les nouvelles sources de données météo pour les mettre à disposition des DS



Assurer la qualité et la disponibilité des données pour les modèles de prévision

SOURCES DE DONNEES - RELEVES METEO

Infoclimat

4 stations
Format Json

Station indépendante 1

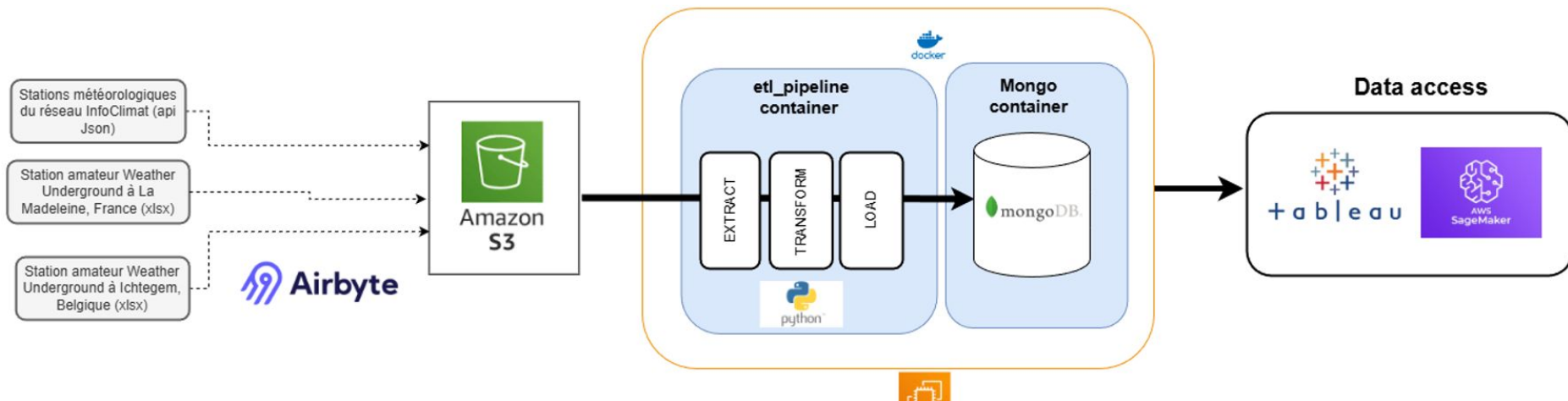
France - La Madeleine
Format Excel

Station indépendante 2

Belgique - Ichtegem
Format Excel

2. Architecture de base de données

Architecture SI



1. Ingestion des données via Airbyte dans un bucket S3.
2. Pipeline ETL : extraction, nettoyage et transformation des données.
3. Stockage dans un MongoDB Container, accessible pour analyses et visualisations (Tableau).
4. Déploiement sur AWS EC2

Etat actuel : pipeline complet, réplication Mongo opérationnelle, Docker déployé sur AWS, orchestrations et transformations en production.

Duplica



[mongo1:27017] ← PRIMARY

|

Replica Set

|

[mongo2:27017] ← SECONDARY

[mongo3:27017] ← SECONDARY

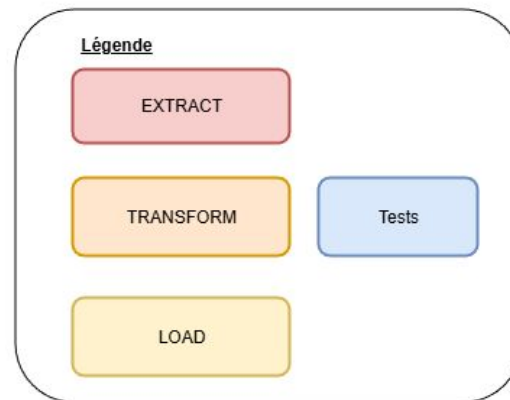
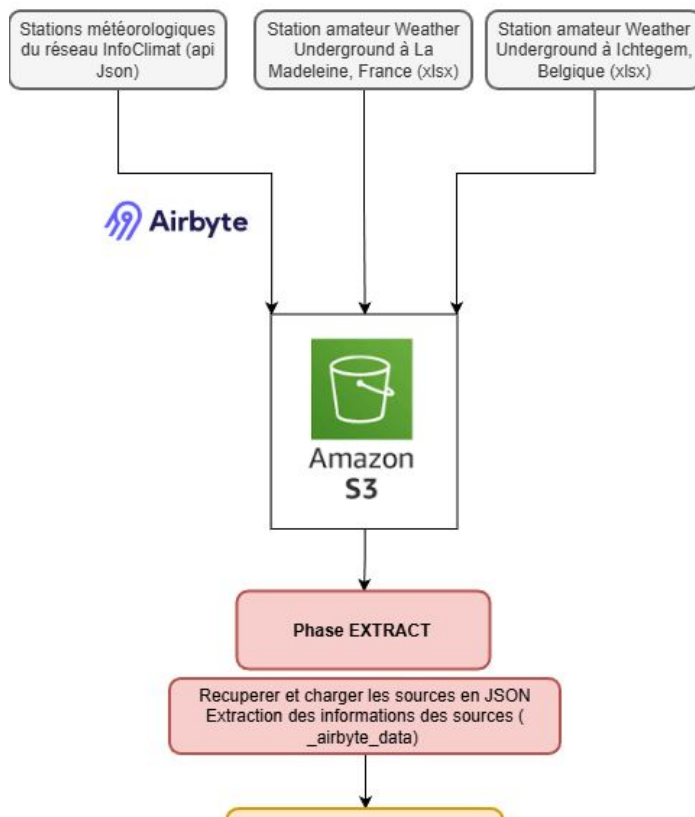
Objectif : garantir la disponibilité
des données.

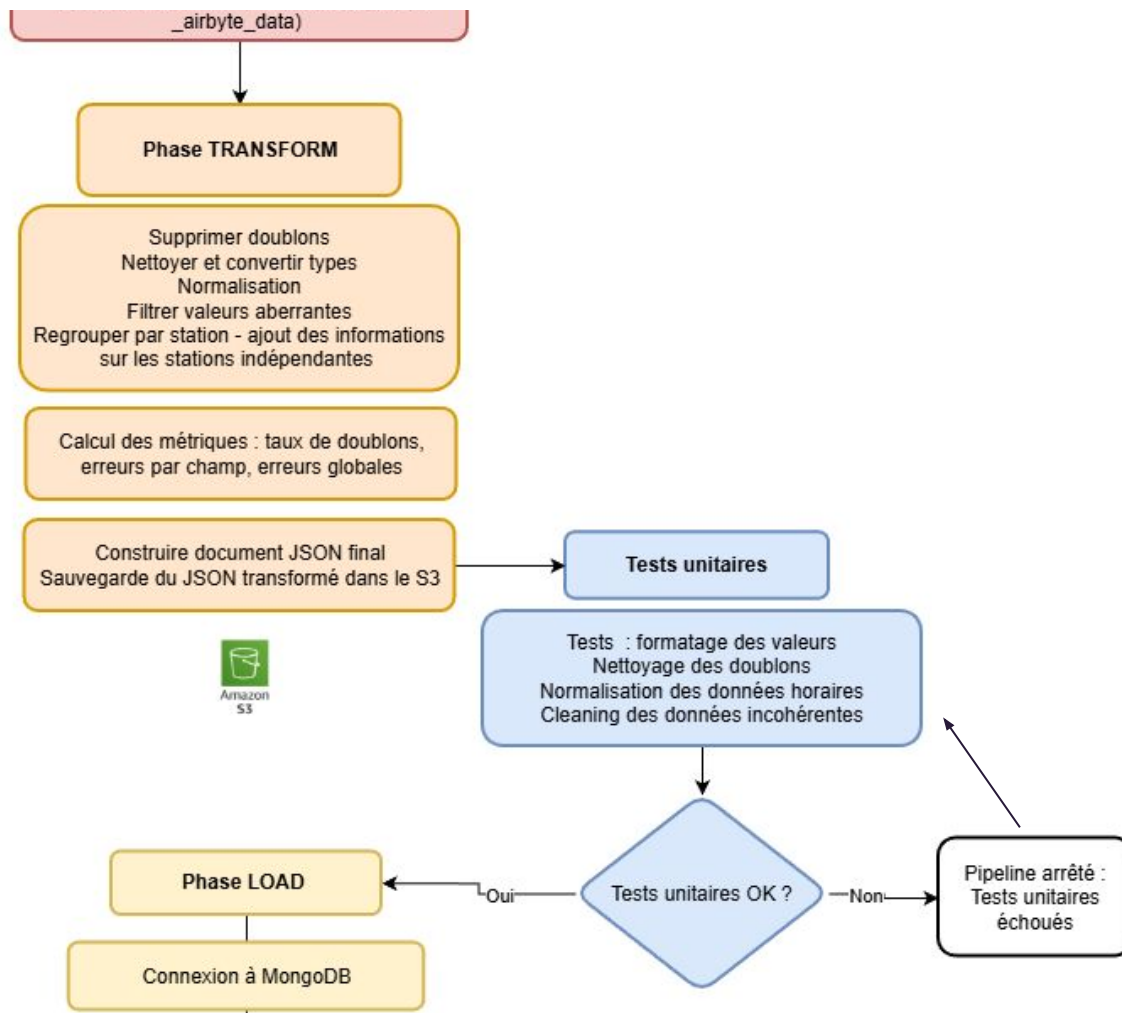
- Si le serveur principal (primary) tombe, un autre prend automatiquement la relève (secondary).

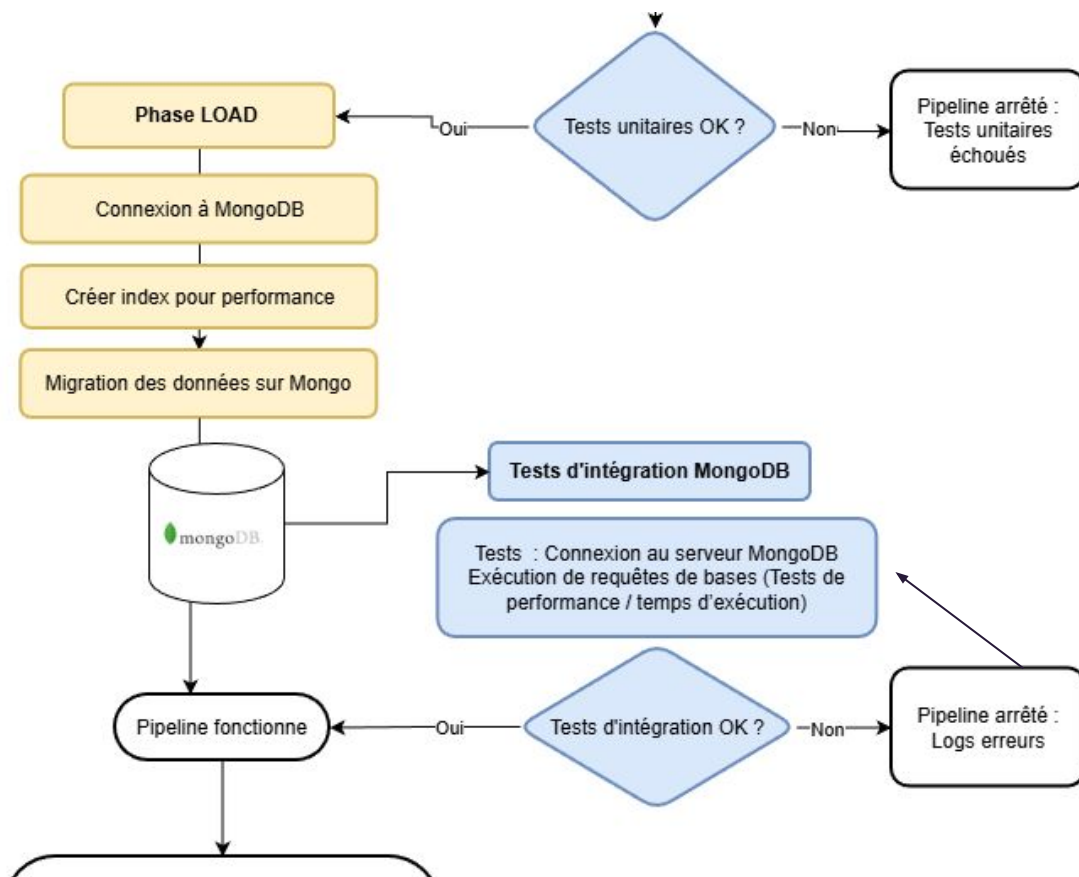
3. Logigramme

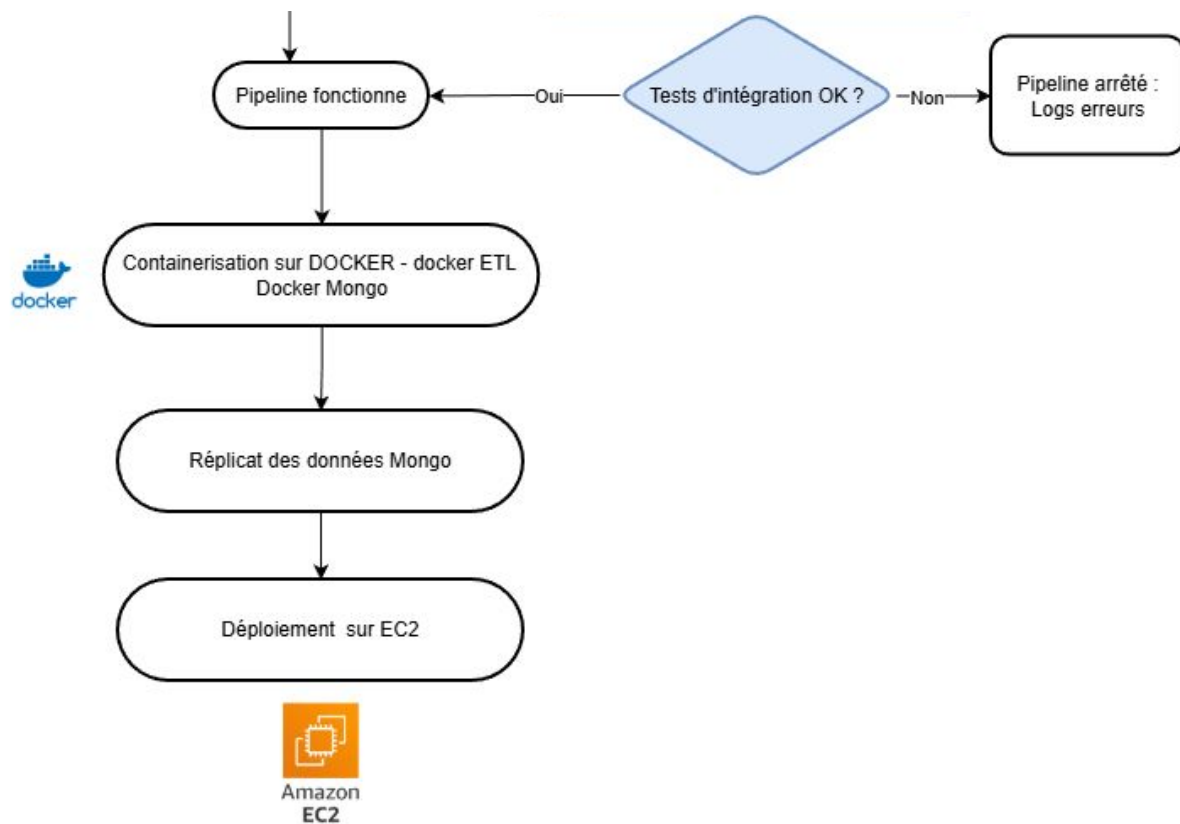
Logigramme ETL

Sources









4. Résultats

Collection

1 document par stations météo

Chaque document contient différentes caractéristiques (40-50 caractéristiques)

Certaines données sont imbriquées (Hourly : relevés des stations) :

Base de données

Collection : meteo_data

Document (station 1)

```
{ "id": "...",  
  "hourly": "..."},...
```

Document (station 2)

```
{ "id": "3"...,  
  "hourly": "..."},...
```

```
{ "_id": "string",  
  "station_id": "string",  
  "name": "string",  
  "type": "string",  
  "source": "string",  
  "latitude": "number",  
  "longitude": "number",  
  "elevation_m": "number",
```

Information sur la stations

```
"hourly": [  
  {"dh_utc": "string (ISO 8601)",  
    "temperature": "number",  
    "pression": "number",  
    ...  
    "neige_au_sol": "number | null",  
    "nebulosite": "number | null",  
    "temps_omm": "string | null" }  
]
```

Données imbriquées (Hourly : relevés des stations)

```
"metadata": {  
  "temperature": "string",  
  "pression": "string",  
  "humidite": "string",  
  ...  
  "nebulosite": "string",  
  "temps_omm": "string" },}}
```

STRUCTURE DU PROJET (accessible sur GIT)

Contenu du projet

```
project/
├── extract_transform/
│   ├── __init__.py           # Fichier d'initialisation du module
│   ├── main_script_extract_transform.py # Script principal pour extraire et transformer les données
│   └── test_extract_transform.py      # Tests unitaires pour l'extraction et la transformation
├── load/
│   ├── load_mongo.py         # Script pour charger les données dans MongoDB
│   └── tests_integration.py   # Tests d'intégration du chargement
├── orchestration/
│   ├── __pycache__/          # Fichiers Python compilés
│   └── run_pipeline.py        # Script pour exécuter l'ensemble du pipeline ETL
├── logs/
│   └── pipeline.log           # Fichier de log généré par le pipeline
├── presentations/            # Contient les documents et visualisations de présentation
├── venv/                     # Environnement virtuel Python
├── .gitignore                # Fichiers et dossiers ignorés par Git
├── docker-compose.yml        # Configuration Docker pour le projet
├── Dockerfile                # Dockerfile pour containeriser l'application
├── init-replica.js           # Script pour initialiser un replica set MongoDB
├── README.md                 # Documentation du projet
└── requirements.txt           # Liste des dépendances Python
```


INJECTION AVEC AIRBYTE DES 3 SOURCES

 Airbyte

WORKSPACE

 Connections

 Sources

 Destinations

 Builder

 Workspace settings

Connections

3 healthy

All connections

All statuses

All sources

All destinations

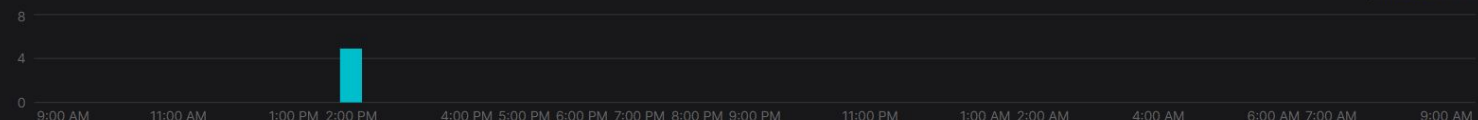
Tags

6h

24h

7d

30d



NAME	SOURCE NAME	DESTINATION NAME	FREQUENCY	TAGS	LAST SYNC	ENABLED
 amateur_bel → S3_meteo_data_json	 amateur_bel	 S3_meteo_data_json	24 hours		19 hours ago	
 amateur_fr → S3_meteo_data_json	 amateur_fr	 S3_meteo_data_json	24 hours		19 hours ago	
 ocr_infoclimat → S3_meteo_data_json	 ocr_infoclimat	 S3_meteo_data_json	24 hours		19 hours ago	

BUCKET S3 AVEC LES RAW



 Rechercher

[Alt+S] 



 Amazon S3 > Compartiments > ocr-projet8 > raw_meteo_data/

Amazon S3

▼ Compartiments

[Compartiments à usage général](#)

Compartiments de répertoires

Compartiments de table

Compartiments de vecteur

▼ Gestion des accès et sécurité

Points d'accès

Points d'accès pour FSx

Access Grants

Analyseur d'accès IAM

▼ Gestion du stockage et informations

Storage Lens

Opérations par lot

Objets (3)

  Copier l'URI S3  Copier l'URL  Télécharger

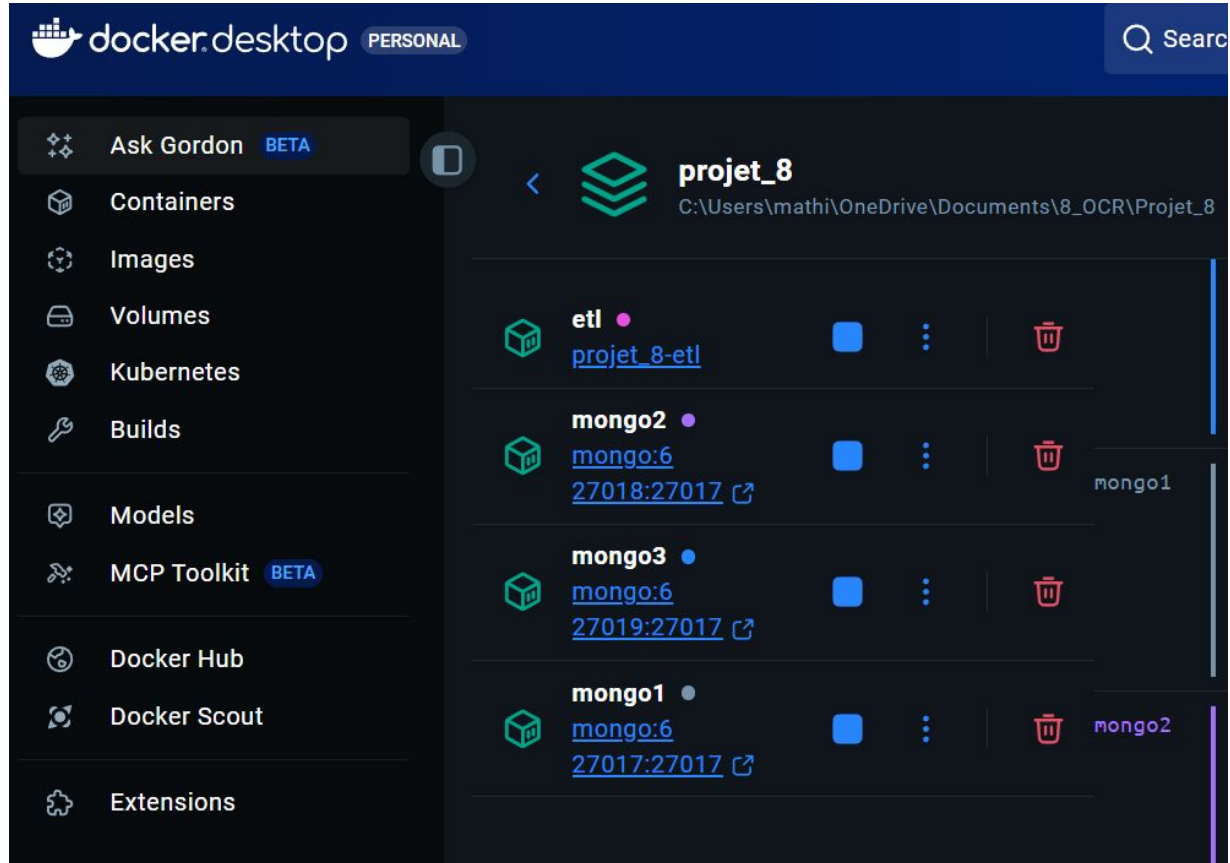
[Créer un dossier](#) [Charger](#)

Les objets sont les entités fondamentales stockées dans Amazon S3. Vous pouvez les objets de votre compartiment. Pour que d'autres personnes puissent accéder autorisations. [En savoir plus](#)

 Rechercher des objets en fonction du préfixe

☐	Nom	Type	Dernière modification
☐	 amateur_bel/	Dossier	-
☐	 amateur_fr/	Dossier	-
☐	 ocr_infoclimat/	Dossier	-

CONTENEURISATION AVEC DOCKER (SCRIPT ETL ET MONGODB)



DÉPLOIEMENT DES CONTENEURS SUR AWS EC2 (UBUNTU)

```
ubuntu@ip-172-31-8-236:~$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
	PORTS		NAMES	
10a885520878	mongo:latest	"docker-entrypoint.s..."	About a minute ago	Up About a minute
	0.0.0.0:27019->27017/tcp, [::]:27019->27017/tcp		mongo3	
c90e73995458	mongo:latest	"docker-entrypoint.s..."	About a minute ago	Up About a minute
	0.0.0.0:27018->27017/tcp, [::]:27018->27017/tcp		mongo2	
2fb7d98e79eb	mongo:latest	"docker-entrypoint.s..."	About a minute ago	Up About a minute
	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp		mongo1	
13f5452e1a6d	projet_8-etl:latest	"python -m orchestra..."	4 minutes ago	Exited (1) 3 minu
tes ago			etl_container	
3329f7d00b19	projet_8-etl:latest	"python -m orchestra..."	25 minutes ago	Exited (1) 24 min
utes ago			boring_sanderson	
c56ddacbc8dd	hello-world	"/hello"	About an hour ago	Exited (0) About
an hour ago			dreamy_williamson	

```
ubuntu@ip-172-31-8-236:~$
```

DÉPLOIEMENT DES CONTENEURS SUR AWS EC2 (INTERFACE AWS)

aws

Rechercher

[Alt+S]

Europe (Paris)

solisdata (7091-845)

EC2 > Instances

EC2

<

Tableau de bord

AWS Global View

Événements

▼ Instances

Instances

Types d'instances

Modèles de lancement

Demandes Spot

Savings Plans

Instances réservées

Hôtes dédiés

Instances (1/1) Informations

Se connecter

État de l'instance

Actions

Lancer des instances

Rechercher Instance par attribut ou identification (case-sensitive)

Tous les états

État de l'instance = running

Effacer les filtres

< 1

☒	Name	ID d'instance	État de l'insta...	Type d'insta...	Contrôle des statu	Sta
☒	Docker p8 meteo_data	i-0137cac77aa76a766	En cours d'...	t3.micro	3/3 vérifications r	Aff

i-0137cac77aa76a766 (Docker p8 meteo_data)

4. Qualité des données obtenues

SUIVI DES LOGS

Notamment, intégrés aux tests d'intégration :
Taux d'erreurs

```
Taux d'erreur :  
Doublons supprimés : 0.00%  
  
Taux de valeurs nulles par champ  
dh_utc: 0 erreurs sur 4950 lignes (0.00%)  
temperature: 0 erreurs sur 4950 lignes (0.00%)  
pression: 0 erreurs sur 4950 lignes (0.00%)  
humidite: 0 erreurs sur 4950 lignes (0.00%)  
point_de_rose: 0 erreurs sur 4950 lignes (0.00%)  
visibilite: 4890 erreurs sur 4950 lignes (98.79%)  
vent_moyen: 0 erreurs sur 4950 lignes (0.00%)  
vent_rafales: 1083 erreurs sur 4950 lignes (21.88%)  
vent_direction: 3830 erreurs sur 4950 lignes (77.37%)  
pluie_1h: 900 erreurs sur 4950 lignes (18.18%)  
pluie_3h: 1052 erreurs sur 4950 lignes (21.25%)  
neige_au_sol: 4950 erreurs sur 4950 lignes (100.00%)  
nebulosite: 4925 erreurs sur 4950 lignes (99.49%)  
  
Taux de valeurs nulles global toutes valeurs confondues : 33.61%  
Hourly regroupées par station  
Documents stations prêts : 6  
JSON sauvegardé avec 6 documents
```


Fin des logs

Tests de performance et temps d'exécution intégrés dans le test d'intégration

```
===== 6 passed in 0.53s =====

Tous les tests sont passés !

Lancement de la pipeline Load...
AWS Key trouvée : Oui
MONGO_URI : mongodb://mongo1:27017/meteo_db
OK : 6 documents chargés depuis S3

Connexion à MongoDB établie.
OK Collection stations_hourly vidée
OK 6 stations insérées avec hourly imbriquées
OK Index créés sur 'id' et 'hourly.dh_utc'

Réussite du Load !

Lancement des tests d'intégration...
Utilisation de l'URI : mongodb://mongo1:27017/meteo_db
Toutes les stations : 0.0431s, 6 documents
Hourly station ILAMAD25 : 0.0164s, 1 documents
Filtre par date : 0.0216s, 6 documents
```

Test temps d'exécution intégrés (extrait du script)

```
# Tests
measure_query_time(lambda: collection.find({}, {"id": 1}), "Toutes les stations")
measure_query_time(lambda: collection.find({"id": "ILAMAD25"}, {"hourly": 1}), "Hourly station ILAMAD25")
measure_query_time(lambda: collection.find({"hourly.dh_utc": {"$gte": "2024-01-23T12:00:00"}}), "Filtre par
```


[CloudWatch](#)[Gestion des journaux](#)[/docker/etl-logs](#)[etl_pipeline](#)

CloudWatch

[Favoris et récents](#)[Ingestion](#)[Tableaux de bord](#)[▶ Alarmes](#) 0 0 0[▶ Signaux d'application \(APM\)](#)[▶ Infrastructure Monitoring](#)[▼ Journaux](#)[Log Management](#) [Nouveau](#)[Anomalies dans les journaux](#)[Queue en direct](#)

Événements de journaux

[Actions](#) ▼[Démarrer le](#)

Vous pouvez utiliser la barre de filtre ci-dessous pour rechercher et faire correspondre des termes, des expressions ou des motifs à un journal. [En savoir plus sur les modèles de filtre](#)

[Affichage](#) ▼**Horodatage****Message**

2026-02-06T09:47:25.973Z

Réussite du Load !



2026-02-06T09:47:25.973Z

Lancement des tests d'intégration...



2026-02-06T09:47:25.973Z

Utilisation de l'URI : mongodb://mongo1:27017/meteo_db



2026-02-06T09:47:25.973Z

Toutes les stations : 0.0431s, 6 documents



2026-02-06T09:47:25.973Z

Hourly station ILAMAD25 : 0.0164s, 1 documents



2026-02-06T09:47:25.973Z

Filtre par date : 0.0216s, 6 documents

Fin du projet projection

Compétences et réalisations :

1. Nettoyage et normalisation de données multi-sources : gestion des unités, suppression des doublons, correction des valeurs incohérentes.
2. Création et structure des scripts
3. Docker et AWS : déploiement d'une application Dockerisée sur une instance EC2 Ubuntu.
4. Haute disponibilité des données : mise en place d'un MongoDB Replica Set.
5. Supervision : gestion des logs et monitoring du pipeline de données.

Suite : orchestration

- Utilisation d'un **scheduler CRON** pour exécuter automatiquement le pipeline :